

CRYPTOGRAPHIC ALGORITHMS

Laboratory Practical Record

Program 1 : Caesar Cipher

Program 2 : Monoalphabetic Cipher

Program 3 : Polyalphabetic (Vigenere) Cipher

Program 4 : Hill Cipher

Program 5 : Playfair Cipher

Program 6 : Rail Fence Cipher

Program 7 : Double Columnar Transposition Cipher

Subject : Network Security & Cryptography

Language : Python 3.x

IDE : Visual Studio Code

Date : September 2026

Program 1: Caesar Cipher

File: 1_caesar_cipher.py

SOURCE CODE

```
# Program 1: Caesar Cipher - Encryption & Decryption
# =====

def caesar_encrypt(plaintext, shift):
    ciphertext = ""
    for ch in plaintext:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            ciphertext += chr((ord(ch) - base + shift) % 26 + base)
        else:
            ciphertext += ch
    return ciphertext

def caesar_decrypt(ciphertext, shift):
    return caesar_encrypt(ciphertext, -shift)

def main():
    test_cases = [
        ("HELLO WORLD", 3),
        ("ATTACK AT DAWN", 13),
        ("Python Programming", 7),
        ("NetworkSecurity2024!", 17),
    ]
    for i, (message, key) in enumerate(test_cases, 1):
        encrypted = caesar_encrypt(message, key)
        decrypted = caesar_decrypt(encrypted, key)
        print(f"--- Test Case {i} ---")
        print(f" Plain Text   : {message}")
        print(f" Shift (Key)  : {key}")
        print(f" Encrypted    : {encrypted}")
        print(f" Decrypted    : {decrypted}")

if __name__ == "__main__":
    main()
```

TERMINAL OUTPUT (VS Code Integrated Terminal)

```
PS C:\crypto_lab> python 1_caesar_cipher.py
=====
                CAESAR CIPHER - Encryption & Decryption
=====

--- Test Case 1 ---
Plain Text   : HELLO WORLD
Shift (Key)  : 3
Encrypted    : KHOOR ZRUOG
Decrypted    : HELLO WORLD

--- Test Case 2 ---
Plain Text   : ATTACK AT DAWN
Shift (Key)  : 13
Encrypted    : NGGNPX NG QNJA
Decrypted    : ATTACK AT DAWN

--- Test Case 3 ---
Plain Text   : Python Programming
Shift (Key)  : 7
Encrypted    : Wfaovu Wyvnyhttpun
Decrypted    : Python Programming

--- Test Case 4 ---
Plain Text   : NetworkSecurity2024!
Shift (Key)  : 17
Encrypted    : EvknfibJvtlizkp2024!
Decrypted    : NetworkSecurity2024!
=====
PS C:\crypto_lab> _
```

Program 2: Monoalphabetic Cipher

File: 2_monoalphabetic_cipher.py

SOURCE CODE

```
# Program 2: Monoalphabetic Cipher - Encryption & Decryption
# =====

ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
CIPHER_KEY = "QWERTYUIOPASDFGHJKLZXCVBNM"

def mono_encrypt(plaintext):
    return "".join(CIPHER_KEY[ALPHABET.index(c)] if c in ALPHABET
                    else c for c in plaintext.upper())

def mono_decrypt(ciphertext):
    return "".join(ALPHABET[CIPHER_KEY.index(c)] if c in CIPHER_KEY
                    else c for c in ciphertext.upper())

def main():
    test_cases = ["HELLO WORLD", "ATTACK AT DAWN",
                  "NETWORK SECURITY", "CRYPTOGRAPHY IS FUN"]
    for i, message in enumerate(test_cases, 1):
        encrypted = mono_encrypt(message)
        decrypted = mono_decrypt(encrypted)
        print(f"--- Test Case {i} ---")
        print(f"  Plain Text   : {message}")
        print(f"  Encrypted    : {encrypted}")
        print(f"  Decrypted    : {decrypted}")

if __name__ == "__main__":
    main()
```

TERMINAL OUTPUT (VS Code Integrated Terminal)

ERONAL 1 bash + x

```
PS C:\crypto_lab> python 2_monoalphabetic_cipher.py
```

```
=====
```

```
MONOALPHABETIC CIPHER - Encryption & Decryption
```

```
=====
```

```
Substitution Key : QWERTYUIOPASDFGHJKLZXCVBNM
```

```
--- Test Case 1 ---
```

```
Plain Text : HELLO WORLD
```

```
Encrypted : ITSSG VGKSR
```

```
Decrypted : HELLO WORLD
```

```
--- Test Case 2 ---
```

```
Plain Text : ATTACK AT DAWN
```

```
Encrypted : QZZQEA QZ RQVF
```

```
Decrypted : ATTACK AT DAWN
```

```
--- Test Case 3 ---
```

```
Plain Text : NETWORK SECURITY
```

```
Encrypted : FTZVGKA LTEXKOZN
```

```
Decrypted : NETWORK SECURITY
```

```
--- Test Case 4 ---
```

```
Plain Text : CRYPTOGRAPHY IS FUN
```

```
Encrypted : EKNHZGUKQHIN OL YXF
```

```
Decrypted : CRYPTOGRAPHY IS FUN
```

```
=====
```

```
PS C:\crypto_lab> _
```

Program 3: Polyalphabetic Cipher

File: 3_polyalphabetic_cipher.py

SOURCE CODE

```
# Program 3: Polyalphabetic (Vigenere) Cipher - Enc & Dec
# =====

def vigenere_encrypt(plaintext, key):
    key = key.upper(); key_index = 0; ciphertext = ""
    for ch in plaintext:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            shift = ord(key[key_index % len(key)]) - ord('A')
            ciphertext += chr((ord(ch) - base + shift) % 26 + base)
            key_index += 1
        else:
            ciphertext += ch
    return ciphertext

def vigenere_decrypt(ciphertext, key):
    key = key.upper(); key_index = 0; plaintext = ""
    for ch in ciphertext:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            shift = ord(key[key_index % len(key)]) - ord('A')
            plaintext += chr((ord(ch) - base - shift) % 26 + base)
            key_index += 1
        else:
            plaintext += ch
    return plaintext

def main():
    test_cases = [("HELLO WORLD", "KEY"), ("ATTACK AT DAWN", "LEMON"),
                  ("NETWORK SECURITY", "CRYPTO"), ("PYTHON IS AMAZING", "SECRET")]
    for i, (message, key) in enumerate(test_cases, 1):
        encrypted = vigenere_encrypt(message, key)
        decrypted = vigenere_decrypt(encrypted, key)
        print(f"--- Test Case {i} ---")
        print(f" Plain Text   : {message}")
        print(f" Key          : {key}")
        print(f" Encrypted    : {encrypted}")
        print(f" Decrypted    : {decrypted}")

if __name__ == "__main__":
    main()
```

TERMINAL OUTPUT (VS Code Integrated Terminal)

TERMINAL 1 bash + x

```
PS C:\crypto_lab> python 3_polyalphabetic_cipher.py
```

```
=====
POLYALPHABETIC (VIGENERE) CIPHER - Enc & Dec
=====
```

```
--- Test Case 1 ---
```

```
Plain Text   : HELLO WORLD
Key          : KEY
Encrypted    : RIJVS UYVJN
Decrypted    : HELLO WORLD
```

```
--- Test Case 2 ---
```

```
Plain Text   : ATTACK AT DAWN
Key          : LEMON
Encrypted    : LXFOPV EF RNHR
Decrypted    : ATTACK AT DAWN
```

```
--- Test Case 3 ---
```

```
Plain Text   : NETWORK SECURITY
Key          : CRYPTO
Encrypted    : PVRLHFM JCRNFKKW
Decrypted    : NETWORK SECURITY
```

```
--- Test Case 4 ---
```

```
Plain Text   : PYTHON IS AMAZING
Key          : SECRET
Encrypted    : HCVYSG AW CDESARI
Decrypted    : PYTHON IS AMAZING
```

```
=====
PS C:\crypto_lab> _
```

Program 4: Hill Cipher

File: 4_hill_cipher.py

SOURCE CODE

```
# Program 4: Hill Cipher - Encryption & Decryption
# =====

def mat_mul_mod(A, v, m=26):
    return [(A[0][0]*v[0]+A[0][1]*v[1])%m,
            (A[1][0]*v[0]+A[1][1]*v[1])%m]

def mod_inv(a, m=26):
    for x in range(1,m):
        if (a*x)%m==1: return x

def mat_inv_2x2(K, m=26):
    det=(K[0][0]*K[1][1]-K[0][1]*K[1][0])%m; di=mod_inv(det,m)
    return [[di*K[1][1]%m, (-di*K[0][1])%m],
            [(-di*K[1][0])%m, di*K[0][0]%m]]

def hill_encrypt(text, K=[[3,3],[2,5]]):
    t = text.upper().replace(" ", "")
    if len(t)%2: t += 'x'
    out = ""
    for i in range(0, len(t), 2):
        v = [ord(t[i])-65, ord(t[i+1])-65]
        r = mat_mul_mod(K, v)
        out += chr(r[0]+65) + chr(r[1]+65)
    return out

def hill_decrypt(text, K=[[3,3],[2,5]]):
    IK = mat_inv_2x2(K); out = ""
    for i in range(0, len(text), 2):
        v = [ord(text[i])-65, ord(text[i+1])-65]
        r = mat_mul_mod(IK, v)
        out += chr(r[0]+65) + chr(r[1]+65)
    return out

def main():
    print(" Key Matrix : [[3, 3], [2, 5]]")
    for i, msg in enumerate(["HELP", "MATH", "PYTHON", "NETWORK"], 1):
        enc = hill_encrypt(msg); dec = hill_decrypt(enc)
        print(f"--- Test Case {i} ---")
        print(f" Plain Text : {msg}")
        print(f" Encrypted : {enc}")
        print(f" Decrypted : {dec}")

if __name__ == "__main__":
    main()
```

TERMINAL OUTPUT (VS Code Integrated Terminal)

```
PS C:\crypto_lab> python 4_hill_cipher.py

=====
HILL CIPHER - Encryption & Decryption
=====

Key Matrix : [[3, 3], [2, 5]]

--- Test Case 1 ---
Plain Text : HELP
Encrypted  : HIAT
Decrypted  : HELP

--- Test Case 2 ---
Plain Text : MATH
Encrypted  : KYAV
Decrypted  : MATH

--- Test Case 3 ---
Plain Text : PYTHON
Encrypted  : NUAVDP
Decrypted  : PYTHON

--- Test Case 4 ---
Plain Text : NETWORK
Encrypted  : ZUTSPJVF
Decrypted  : NETWORKX

=====
PS C:\crypto_lab> _
```

Program 5: Playfair Cipher

File: 5_playfair_cipher.py

SOURCE CODE

```
# Program 5: Playfair Cipher - Encryption & Decryption
# =====

def generate_matrix(key):
    seen = []
    for c in key.upper().replace("J","I")+"ABCDEFGHIKLMNOPQRSTUVWXYZ":
        if c not in seen: seen.append(c)
    return [seen[i:i+5] for i in range(0,25,5)]

def find_pos(M, c):
    for r,row in enumerate(M):
        if c in row: return r, row.index(c)

def prepare(text):
    t = text.upper().replace("J","I").replace(" ", ""); res=[]; i=0
    while i < len(t):
        a = t[i]; b = t[i+1] if i+1<len(t) else 'X'
        if a==b: res.append((a,'X')); i+=1
        else: res.append((a,b)); i+=2
    return res

def playfair_encrypt(text, key):
    M = generate_matrix(key); out = ""
    for a,b in prepare(text):
        r1,c1 = find_pos(M,a); r2,c2 = find_pos(M,b)
        if r1==r2: out += M[r1][[(c1+1)%5]+M[r2][[(c2+1)%5]
        elif c1==c2: out += M[(r1+1)%5][c1]+M[(r2+1)%5][c2]
        else: out += M[r1][c2]+M[r2][c1]
    return out

def playfair_decrypt(text, key):
    M = generate_matrix(key); out = ""
    for i in range(0,len(text),2):
        a,b = text[i],text[i+1]
        r1,c1 = find_pos(M,a); r2,c2 = find_pos(M,b)
        if r1==r2: out += M[r1][[(c1-1)%5]+M[r2][[(c2-1)%5]
        elif c1==c2: out += M[(r1-1)%5][c1]+M[(r2-1)%5][c2]
        else: out += M[r1][c2]+M[r2][c1]
    return out

def main():
    cases = [("HELLO","KEYWORD"),("MEET ME","MONARCHY"),
             ("HIDE GOLD","PLAYFAIR"),("ATTACK NOW","SECRET")]
    for i,(msg,key) in enumerate(cases,1):
        enc=playfair_encrypt(msg,key); dec=playfair_decrypt(enc,key)
        print(f"--- Test Case {i} ---")
        print(f" Plain Text   : {msg}")
        print(f" Key           : {key}")
        print(f" Encrypted     : {enc}")
        print(f" Decrypted     : {dec}")

if __name__ == "__main__":
    main()
```

TERMINAL OUTPUT (VS Code Integrated Terminal)

```
PS C:\crypto_lab> python 5_playfair_cipher.py
=====
PLAYFAIR CIPHER - Encryption & Decryption
=====

--- Test Case 1 ---
Plain Text   : HELLO
Key          : KEYWORD
Encrypted    : GYIZSC
Decrypted    : HELXLO

--- Test Case 2 ---
Plain Text   : MEET ME
Key          : MONARCHY
Encrypted    : CLKLCL
Decrypted    : MEETME

--- Test Case 3 ---
Plain Text   : HIDE GOLD
Key          : PLAYFAIR
Encrypted    : EBIMOVFR
Decrypted    : HIDEGOLD

--- Test Case 4 ---
Plain Text   : ATTACK NOW
Key          : SECRET
Encrypted    : GSSGDPOPHY
Decrypted    : ATTACKNOWX
=====
PS C:\crypto_lab> _
```

Program 6: Rail Fence Cipher

File: 6_rail_fence_cipher.py

SOURCE CODE

```
# Program 6: Rail Fence Cipher - Encryption & Decryption
# =====

def rail_fence_encrypt(text, rails):
    fence = [[] for _ in range(rails)]; rail=0; direction=1
    for ch in text:
        fence[rail].append(ch)
        if rail==0: direction=1
        elif rail==rails-1: direction=-1
        rail += direction
    return "".join("".join(r) for r in fence)

def rail_fence_decrypt(text, rails):
    n = len(text); pat=[]; rail=0; d=1
    for _ in range(n):
        pat.append(rail)
        if rail==0: d=1
        elif rail==rails-1: d=-1
        rail += d
    idx = sorted(range(n), key=lambda i: pat[i])
    res = ['']*n
    for i,j in enumerate(idx): res[j] = text[i]
    return "".join(res)

def main():
    cases = [("HELLO WORLD",3),("ATTACK AT DAWN",2),
             ("WEAREDISCOVEREDRUNATONCE",3),("NETWORK SECURITY IS KEY",4)]
    for i,(msg,rails) in enumerate(cases,1):
        enc = rail_fence_encrypt(msg,rails)
        dec = rail_fence_decrypt(enc,rails)
        print(f"--- Test Case {i} ---")
        print(f"  Plain Text   : {msg}")
        print(f"    Rails      : {rails}")
        print(f"  Encrypted    : {enc}")
        print(f"  Decrypted    : {dec}")

if __name__ == "__main__":
    main()
```

TERMINAL OUTPUT (VS Code Integrated Terminal)

TERMINAL 1 bash + x

PS C:\crypto_lab> python 6_rail_fence_cipher.py

```
=====
      RAIL FENCE CIPHER - Encryption & Decryption
=====
```

--- Test Case 1 ---

```
Plain Text   : HELLO WORLD
Rails        : 3
Encrypted    : HOREL OLLWD
Decrypted    : HELLO WORLD
```

--- Test Case 2 ---

```
Plain Text   : ATTACK AT DAWN
Rails        : 2
Encrypted    : ATC TDWTAKA AN
Decrypted    : ATTACK AT DAWN
```

--- Test Case 3 ---

```
Plain Text   : WEAREDISCOVEREDRUNATONCE
Rails        : 3
Encrypted    : WECRUOERDSOEERNTNEAIVDAC
Decrypted    : WEAREDISCOVEREDRUNATONCE
```

--- Test Case 4 ---

```
Plain Text   : NETWORK SECURITY IS KEY
Rails        : 4
Encrypted    : NKRSER UII TOSCT KYWEYE
Decrypted    : NETWORK SECURITY IS KEY
```

```
=====
PS C:\crypto_lab> _
```

Program 7: Double Columnar Transposition

File: 7_double_columnar_cipher.py

SOURCE CODE

```
# Program 7: Double Columnar Transposition Cipher
# =====

def col_order(key):
    ix = sorted(enumerate(key.upper()), key=lambda x:x[1])
    o = [0]*len(key)
    for rank,(oi,_) in enumerate(ix): o[oi]=rank
    return o

def col_encrypt(text, key):
    c = len(key)
    while len(text)%c: text += 'X'
    rows = len(text)//c
    grid = [list(text[i*c:(i+1)*c]) for i in range(rows)]
    od = col_order(key); seq = sorted(range(c), key=lambda x:od[x])
    return "".join(grid[r][col] for col in seq for r in range(rows))

def col_decrypt(text, key):
    c=len(key); rows=len(text)//c
    od=col_order(key); seq=sorted(range(c),key=lambda x:od[x])
    grid=[['']*c for _ in range(rows)]; idx=0
    for col in seq:
        for r in range(rows): grid[r][col]=text[idx]; idx+=1
    return "".join("".join(row) for row in grid)

def double_encrypt(text, k1, k2):
    return col_encrypt(col_encrypt(text.upper().replace(" ", ""),k1),k2)

def double_decrypt(text, k1, k2):
    return col_decrypt(col_decrypt(text,k2),k1)

def main():
    cases=[("HELLO WORLD", "SECRET", "KEY"), ("ATTACK AT DAWN", "CRYPTO", "LOCK"),
           ("NETWORK SECURITY", "ALPHA", "BETA"), ("WE ARE DISCOVERED RUN", "ZEBRA", "MAGIC")]
    for i,(msg,k1,k2) in enumerate(cases,1):
        enc=double_encrypt(msg,k1,k2); dec=double_decrypt(enc,k1,k2)
        print(f"--- Test Case {i} ---")
        print(f" Plain Text   : {msg}")
        print(f" Key 1        : {k1}")
        print(f" Key 2        : {k2}")
        print(f" Encrypted    : {enc}")
        print(f" Decrypted    : {dec}")

if __name__ == "__main__":
    main()
```

TERMINAL OUTPUT (VS Code Integrated Terminal)

ERONAL 1 bash + x

PS C:\crypto_lab> python 7_double_columnar_cipher.py

=====

DOUBLE COLUMNAR TRANSPOSITION - Encryption & Decryption

=====

--- Test Case 1 ---

Plain Text : HELLO WORLD

Key 1 : SECRET

Key 2 : KEY

Encrypted : LODWLRLOEXHX

Decrypted : HELLOWORLDXX

--- Test Case 2 ---

Plain Text : ATTACK AT DAWN

Key 1 : CRYPTO

Key 2 : LOCK

Encrypted : KTTNTDAACAAW

Decrypted : ATTACKATDAWN

--- Test Case 3 ---

Plain Text : NETWORK SECURITY

Key 1 : ALPHA

Key 2 : BETA

Encrypted : OERXNCTTRYESUWKI

Decrypted : NETWORKSECURITY

--- Test Case 4 ---

Plain Text : WE ARE DISCOVERED RUN

Key 1 : ZEBRA

Key 2 : MAGIC

Encrypted : ORUWAIERDNRDXECVESEX

Decrypted : WEAREDISCOVEREDRUNXX

=====

PS C:\crypto_lab> _